

Experiment 'p_4_3_b_every_second_letter' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_3_b_every_second_letter

Natural language statement: null

Method signature: p_4_3_b_every_second_letter(s: string) returns (result: string)

Ensures

- $|result| * 2 == |s| \ || \ |result| * 2 + 1 == |s|$
- forall $i :: 0 \leq i < |result| \implies result[i] == s[2 * i]$

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_4_3_b_every_second_letter(s: string) returns (result: string)`

The method should respect the following contract:

`ensures |result| * 2 == |s| || |result| * 2 + 1 == |s|, ensures forall i :: 0 <= i < |`
`↪ result| ==> result[i] == s[2 * i]`

Produce and show only the Dafny body of this method, including the curly braces that
`↪` surround it. Do not show the signature nor contract. You must not use 'assume' nor
`↪` 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLpTZCynPpxiXXhv0ssNdXMTrrh6

Model: gpt-5.5-2026-04-23

Created at: 1780654355

Finish reason: stop

Usage: 332 tokens in, and 4178 tokens out

```
{
  result := "";
  var k := 0;

  while k + 1 < |s|
    invariant 0 <= k <= |s|
    invariant k == 2 * |result|
    invariant forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
    decreases |s| - k
  {
    var oldResult := result;
    var oldK := k;

    assert forall i :: 0 <= i < |oldResult| ==> oldResult[i] == s[2 * i];
    assert oldK + 1 <= |s|;

    result := oldResult + s[oldK..oldK + 1];

    assert |s[oldK..oldK + 1]| == 1;
    assert |result| == |oldResult| + 1;
```

```

    assert result[|oldResult|] == (s[oldK..oldK + 1])[0];
    assert (s[oldK..oldK + 1])[0] == s[oldK];
    assert result[|oldResult|] == s[oldK];

    assert forall i :: 0 <= i < |result| ==> result[i] == s[2 * i] by {
      forall i | 0 <= i < |result|
        ensures result[i] == s[2 * i]
      {
        if i < |oldResult| {
          assert result[i] == oldResult[i];
          assert oldResult[i] == s[2 * i];
        } else {
          assert i == |oldResult|;
          assert oldK == 2 * |oldResult|;
          assert oldK == 2 * i;
          assert result[i] == s[oldK];
        }
      }
    }

    k := oldK + 2;
    assert k == 2 * |result|;
    assert k <= |s|;
  }

  assert |s| <= k + 1;
  if k == |s| {
    assert |result| * 2 == |s|;
  } else {
    assert k < |s|;
    assert k + 1 == |s|;
    assert |result| * 2 + 1 == |s|;
  }
}

```

Final Program

```

method p_4_3_b_every_second_letter(s: string) returns (result: string)
  ensures |result| * 2 == |s| || |result| * 2 + 1 == |s|
  ensures forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
{
  result := "";
  var k := 0;

  while k + 1 < |s|
    invariant 0 <= k <= |s|
    invariant k == 2 * |result|
    invariant forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
    decreases |s| - k
  {

```

```

var oldResult := result;
var oldK := k;

assert forall i :: 0 <= i < |oldResult| ==> oldResult[i] == s[2 * i];
assert oldK + 1 <= |s|;

result := oldResult + s[oldK..oldK + 1];

assert |s[oldK..oldK + 1]| == 1;
assert |result| == |oldResult| + 1;
assert result[|oldResult|] == (s[oldK..oldK + 1])[0];
assert (s[oldK..oldK + 1])[0] == s[oldK];
assert result[|oldResult|] == s[oldK];

assert forall i :: 0 <= i < |result| ==> result[i] == s[2 * i] by {
  forall i | 0 <= i < |result|
    ensures result[i] == s[2 * i]
  {
    if i < |oldResult| {
      assert result[i] == oldResult[i];
      assert oldResult[i] == s[2 * i];
    } else {
      assert i == |oldResult|;
      assert oldK == 2 * |oldResult|;
      assert oldK == 2 * i;
      assert result[i] == s[oldK];
    }
  }
}

k := oldK + 2;
assert k == 2 * |result|;
assert k <= |s|;
}

assert |s| <= k + 1;
if k == |s| {
  assert |result| * 2 == |s|;
} else {
  assert k < |s|;
  assert k + 1 == |s|;
  assert |result| * 2 + 1 == |s|;
}
}

```

Total Token Usage

Input tokens: 332

Output tokens: 4178

Reasoning tokens: 3583

Sum of ‘total tokens’: 4510

Experiment Timings

Overall Experiment started at 1780654355180, ended at 1780654449655, lasting 94475ms (94.48 seconds)

Iteration #1 started at 1780654355181, ended at 1780654449655, lasting 94474ms (94.47 seconds)